

Lab 6 Challenge

Scenario

You have been asked to build a **Game Tournament Analytics System**.

A gaming tournament has finished and the organizers have collected information about players, teams, scores, matches and countries.

Unfortunately, some of the data is inconsistent and difficult to analyze.

Your task is to clean, transform, combine, sort and present the tournament data using the Pythonic techniques covered in Lesson 6.

Unlike the standard lab, you will not be told exactly where to use each technique. You are expected to decide when comprehensions, enumerate(), zip(), unpacking, sorted() and lambda expressions make the code clearer.

Part 1 - Tournament data

Create data representing at least **15 players**.

Each player should contain information such as:

- player name
- team
- country
- score
- matches played
- wins
- active status

Make some of the original data deliberately inconsistent.

For example:

- extra whitespace in names
- inconsistent capitalization
- duplicate country names with different casing
- inactive players
- players with zero wins
- different scores

Your data should give you something meaningful to clean and analyze.

Part 2 - Clean the player data

Create a cleaned version of the tournament data.

Normalize appropriate text values so that information such as:

```
"  anna  "  
"ANNA"
```

"anna"

is represented consistently.

Do not modify the original data.

Your cleaned data should be stored separately.

Use comprehensions where they make the transformation clearer.

Part 3 - Filter the tournament

Create separate collections representing:

- active players
- players with at least 3 wins
- players with a score above a threshold of your choice
- players from a specific country
- players who meet **two conditions at the same time**

Do not manually add players to these collections.

Think about when a comprehension provides a cleaner solution than a normal loop.

Part 4 - Tournament statistics

Use the cleaned data to create:

- a collection of all unique countries
- a collection of all unique teams
- a dictionary mapping player names to scores
- a dictionary mapping player names to number of wins
- a dictionary containing only players who reached a score threshold

At least one of these should use a **set comprehension** and at least two should use **dictionary comprehensions**.

Part 5 - Combine tournament data

The organizers also have some information stored in separate lists.

For example:

```
player_names = ["Anna", "David", "Sara", "Leo"]
```

```
ranking_points = [1200, 950, 1430, 1100]
```

Create at least **three meaningful examples** where separate collections are combined.

At least one example should combine **three collections**.

Use `zip()` and unpack the resulting values appropriately.

Also investigate what happens if the collections passed to `zip()` have different lengths.

Add a short comment explaining the result.

Part 6 - Ranking system

Create tournament rankings based on the cleaned player data.

Produce rankings for at least:

- highest score
- most wins
- most matches played
- player name alphabetically

Use `sorted()` with appropriate sorting keys.

At least two rankings should use a lambda.

Create both an ascending and descending ranking somewhere in your solution.

The original player collection should remain unchanged.

Part 7 - Ranked tournament report

Create a readable leaderboard showing the players in ranking order.

The output should look conceptually like:

TOURNAMENT LEADERBOARD

1. Sara - 1430 points
2. Anna - 1200 points
3. Leo - 1100 points
4. David - 950 points

Do not manually create the ranking numbers.

Use a Python feature from today's lesson to generate them.

Part 8 - Team analysis

Use the tournament data to answer questions such as:

- Which players belong to a particular team?
- Which players have more than a chosen number of wins?
- Which teams are represented in the tournament?
- Which countries are represented?
- Which active players have reached a chosen score?

Create at least **five useful derived collections** from the original tournament data.

Choose whether each result should be represented as a:

- list
- set
- dictionary

Your choice should make sense for the information being represented.

Part 9 - Player performance

Create a new collection where each player has a calculated performance value.

You decide how the performance value should be calculated.

For example, you could consider:

- score
- wins
- matches played

Then:

- rank players by performance
- find the top performers
- create a collection containing only players above a chosen performance threshold
- create a dictionary mapping player names to performance values

Try to express these transformations clearly and concisely.

Final Challenge - Tournament Analytics Report

Create a final report containing at least:

- total number of players
- number of active players
- unique teams
- unique countries
- players ranked by score
- players ranked by wins
- top 5 players
- players above your performance threshold

Also add at least **three additional analyses of your own choice**.

Examples could include:

- undefeated players
- players with no wins
- highest-performing team
- players from each country
- average score comparison
- players who satisfy several conditions simultaneously

Present the results in a readable way.

Use `enumerate()` where numbered output improves readability.

Pythonic Design Challenge

Review your finished solution.

Find at least **three places** where you originally could have used a normal loop but chose a comprehension or another Pythonic technique instead.

For each example, explain briefly:

- What the longer solution would have looked like.
- What Pythonic technique you used.
- Why your final version is clearer.

Then create **one deliberately over-complicated comprehension**.

Rewrite it using a clearer approach.

Add a comment explaining why the second version is better.

Remember: Shorter code is not automatically better code.

The goal is readable and expressive Python.

Requirements

Your solution must demonstrate meaningful use of:

- list comprehensions
- dictionary comprehensions
- set comprehensions
- conditional filtering in comprehensions
- enumerate()
- zip()
- unpacking
- sorted()
- sorting with key
- lambda expressions

Your program should:

- preserve the original raw data when cleaning it
- use descriptive variable names
- avoid unnecessary repetition
- use comprehensions only when they remain readable
- create meaningful derived data rather than artificial examples
- run without errors

Restrictions

Do not use:

- classes
- file handling
- external libraries
- Pandas

- NumPy

These concepts belong to later parts of the program.

You are **not required to replace every loop with a comprehension**. Part of this challenge is demonstrating that you understand when Pythonic syntax improves readability and when a normal loop is the better choice.